Python Code

```
import numpy as np
import matplotlib.pyplot as plt

class Ga:
    def __init__(self, fitness_func, inf_limit, sup_limit, pop_size, genes, MUT_FACTOR=0.2, CROSS_FACTOR=0.7, custo=True, GERACOES = 300, Int_alg=False):
        if not Int_alg:
            self.pop = np.random.uniform(low=inf_limit, high=sup_limit, size=(pop_size, genes))
        else:
            self.pop = np.random.randint(sup_limit, size=(pop_size, genes))
        #EIXOS DO GRAFICO
        self.fit = fitness_func
        self.fitness_por_geracao = np.zeros(shape=GERACOES, dtype=float)
        self.geracoes = np.linspace(start=1, stop=GERACOES, num=GERACOES, dtype=int)
        print(self.pop)
        for _ in range(GERACOES):
            print(_)
            if not Int_alg:
                new_pop_one = np.zeros((pop_size, genes), float)
                new_pop_two = np.zeros((pop_size, genes), float)

            else:
                new_pop_one = np.zeros((pop_size, genes), int)
                new_pop_two = np.zeros((pop_size, genes), float)


            for i in range(pop_size): #Seleção

                first = self.pop[np.random.randint(pop_size)]
                # print(first)
                second = self.pop[np.random.randint(pop_size)]
                # print(second)
                #Definimos aqui os fitness, seleção Feita!
                if custo:
                    if fitness_func(first) > fitness_func(second):
                        new_pop_one[i] = second
                    else:
                        new_pop_one[i] = first

                else:
                    if fitness_func(second) > fitness_func(first):
                        new_pop_one[i] = second
                    else:
                        new_pop_one[i] = first

            self.pop = new_pop_one
            new_pop_size = 0

```

```
            while new_pop_size < pop_size: #Cruzamento

                first_father = self.pop[np.random.randint(pop_size)]
                second_father = self.pop[np.random.randint(pop_size)]
                cross_p = np.random.choice([0, 1], p=(1 - CROSS_FACTOR,
CROSS_FACTOR))

                if cross_p == 0:
                    new_pop_two[new_pop_size] = first_father
                    new_pop_two[new_pop_size + 1] = second_father
                    new_pop_size += 2
                    continue

                if not Int_alg: #Aquela sacanagem do não inteiro

                    rng = np.random.default_rng()
                    beta = rng.normal(loc=0.0, scale=1.0)
                    first_son = beta * first_father + (1 - beta) *
second_father
                    second_son = beta * second_father + (1 - beta) *
first_father
                    new_pop_two[new_pop_size] = np.clip(first_son, inf_limit,
sup_limit)
                    new_pop_two[new_pop_size + 1] = np.clip(second_son,
inf_limit, sup_limit)
                    new_pop_size += 2
                    continue

                else:
                    break_idx = np.random.randint(pop_size) #Posicao de quebra
                    new_pop_two[new_pop_size] =
np.concatenate((first_father[break_idx:], second_father[:break_idx]))
                    new_pop_two[new_pop_size + 1] =
np.concatenate((second_father[break_idx:], first_father[:break_idx]))
                    new_pop_size += 2
                    continue

            self.pop = new_pop_two

        #mutacao
            for i in range(pop_size):
                    cross_m = np.random.choice([0, 1], p=(1 - MUT_FACTOR,
MUT_FACTOR))
                    j = np.random.randint(0, genes)
                    if not cross_m:
                        continue

                    if Int_alg:
                        self.pop[i][j] = np.random.randint(inf_limit,
sup_limit)
                        continue

                    self.pop[i][j] = np.random.uniform(inf_limit, sup_limit)
```

```python
            fitness_arr = np.array([fitness_func(ind) for ind in self.pop])
            melhor_idx = np.argmin(fitness_arr)
            self.fitness_por_geracao[_] = fitness_arr[melhor_idx]

    def best(self):
        return sorted(self.pop, key=self.fit)[0]

    def plot(self):
        fig, ax = plt.subplots()

        ax.plot(self.geracoes, self.fitness_por_geracao)

        ax.set_xlabel("Geração")
        ax.set_ylabel("Fitness")
        ax.set_title("Algoritmo GA")
        ax.legend(["Fitness"])

        plt.show()
```